

Node.js - Websockets

Bisher haben wir in unseren Node.js-Anwendungen hauptsächlich HTTP verwendet, um Daten zwischen dem Client und dem Server auszutauschen. HTTP ist jedoch ein zustandsloses Protokoll, was bedeutet, dass jede Anfrage unabhängig von vorherigen Anfragen behandelt wird. Dies kann in bestimmten Szenarien, wie z.B. bei Echtzeitanwendungen, unpraktisch sein.

Websockets bieten eine Lösung für dieses Problem, indem sie eine bidirektionale Kommunikationsverbindung zwischen dem Client und dem Server ermöglichen. Dies bedeutet, dass sowohl der Client als auch der Server jederzeit Nachrichten senden können, ohne dass eine neue Anfrage gestellt werden muss.

Socket.IO

Socket.IO ist eine beliebte Bibliothek, die die Implementierung von Websockets in Node.js-Anwendungen erleichtert. Sie bietet eine einfache API und unterstützt verschiedene Transportmechanismen, um eine stabile Verbindung zu gewährleisten.

Installation

Um Socket.IO in deinem Node.js-Projekt zu verwenden, musst du es zuerst installieren. Du kannst dies mit **npm** tun:

```
npm install socket.io
```

Einfache Websocket-Anwendung

Server-Seite

Erstellen wir zur Demonstration ein neues Node.js-Projekt, das Websockets verwendet, um Nachrichten zwischen dem Client und dem Server auszutauschen. Dazu implementieren wir zuerst die Server-Seite.

```
mkdir ws_server
cd ws_server
npm init -y
npm install express socket.io
```

Öffne das Projekt in VS Code und stelle das Modul-System auf "module" um, indem du die `package.json`-Datei anpasst:

```
{
  "type": "module",
  ...
}
```

Erstelle eine `index.js`-Datei mit folgendem Inhalt:

```
import express from 'express';
import { createServer } from 'http';
import { Server } from 'socket.io';

const PORT = 3000;

const app = express();

const httpServer = createServer(app);
```

```
const io = new Server(httpServer);

io.on('connection', (socket) => {
  console.log(`new client connected: ${socket.id}`);

  socket.on('disconnect', () => {
    console.log(`client with id ${socket.id} has disconnected`);
  });
});

httpServer.listen(PORT, () => {
  console.log(`backend server listening of port ${PORT}`);
});
```

Passe nun das `package.json`-Skript an, um den Server zu starten:

```
"scripts": {
  "start": "node --watch index.js"
}
```

Starte den Backend-Server mit **npm start**.

Client-Seite

Auf Client-Seite verwenden wir EJS als Template-Engine, um eine einfache HTML-Seite zu rendern, die mit dem Websocket-Server kommuniziert.

```
mkdir ws_client
cd ws_client
npm init -y
npm install express socket.io
```

Stelle auch hier das Modul-System auf "module" um und erstelle ein "start"-Script.

Erstelle nun eine `index.js`-Datei mit folgendem Inhalt:

```
import express from 'express';

const PORT = 3100;

const app = express();
app.set('view engine', 'ejs');

app.get('/', (req, res) => {
  res.render('index');
});

app.listen(PORT, () => {
  console.log(`frontend server listening on port ${PORT}`);
});
```

Erstelle schließlich noch ein `views`-Verzeichnis und darin eine `index.ejs`-Datei mit folgendem Inhalt:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
    ↪ initial-scale=1.0">
  <title>WS Demo</title>
  <script
    ↪ src="http://localhost:3000/socket.io/socket.io.js"></script>
</head>
```

```
<body>
  <h1>WS Demo</h1>
  <hr>
  <input id="msg" placeholder="Nachricht eingeben">
  <button id="sendBtn">Senden</button>
  <script>
    const socket = io('http://localhost:3000');

    document.getElementById('sendBtn').addEventListener('click', ()
    ↪ => {
      console.log('send button clicked');
      const msg = document.getElementById('msg').value;
      console.log(`msg = "${msg}"`);
    });
  </script>
</body>

</html>
```

Starte nun auf den Frontend-Server mit **npm start** und öffne die Seite im Browser unter `http://localhost:3100`.

Jetzt wirst Du eine **CORS**-Fehlermeldung in der Browser-Konsole sehen, da der Websocket-Server auf Port 3000 läuft und der Client auf Port 3100. Um dies zu beheben, fügen wir dem Backend-Server CORS-Unterstützung hinzu.

CORS-Unterstützung

Ändere den Code in der **index.js**-Datei des Backend-Servers wie folgt:

```
const io = new Server(httpServer, {
  cors: {
```

```
    origin: '*',  
    methods: ['GET', 'POST']  
  }  
});
```

Im Frontend-Projekt installieren wir noch das CORS-Paket:

```
npm install cors
```

Aktiviere noch diese Express-Middleware in der **index.js**-Datei des Frontend-Servers:

```
import express from 'express';  
import cors from 'cors'; // <-- CORS Middleware importieren  
  
const PORT = 3100;  
  
const app = express();  
app.set('view engine', 'ejs');  
  
app.use(cors()); // <-- CORS Middleware aktivieren  
  
app.get('/', (req, res) => {  
  res.render('index');  
});  
  
app.listen(PORT, () => {  
  console.log(`frontend server listening on port ${PORT}`);  
});
```

Verbindung testen

Nun solltest Du in der Konsole vom Backend-Server sehen, dass ein neuer Client verbunden ist:

```
backend server listening of port 3000  
new client connected: rTho3CLmElaN1Ds2AAAB  
new client connected: WYzEY62I_FJZKeRVAAAD
```

Nachrichten senden

Um Nachrichten zwischen dem Client und dem Server zu senden, erweitern wir den Code auf beiden Seiten.

Füge in **index.ejs** folgenden Code im `<script>`-Block hinzu, um Nachrichten an den Server zu senden:

```
...  
<script>  
  const socket = io('http://localhost:3000');  
  
  document.getElementById('sendBtn').addEventListener('click', () =>  
    ↪ {  
      console.log('send button clicked');  
      const msg = document.getElementById('msg').value;  
      console.log(`msg = "${msg}"`);  
      socket.emit('new-message', msg); // <-- Nachricht an den Server  
      ↪ senden  
    });  
  
    // Nachricht vom Server empfangen  
    socket.on('new-message', (msg) => {  
      console.log(`message received: "${msg}"`);  
    });  
</script>  
...
```

Wir senden also eine Nachricht mit dem Event-Namen `new-message` an den Server. Auf der Server-Seite fügen wir folgenden Code in **index.js** hinzu, um die Nachricht zu empfangen und an alle verbundenen Clients zu senden:

```
io.on('connection', (socket) => {  
  console.log(`new client connected: ${socket.id}`);  
  
  // Nachricht vom Client empfangen  
  socket.on('new-message', (msg) => {  
    console.log(`new message received: "${msg}"`);  
    io.emit('new-message', `** ${msg} **`); // an alle Clients  
    // socket.emit(...) Nachricht nur an den Sender  
  });  
  
  socket.on('disconnect', () => {  
    console.log(`client with id ${socket.id} has disconnected`);  
  });  
});
```

Jetzt kannst du eine Nachricht im Eingabefeld eingeben und auf "Senden" klicken. Die Nachricht wird an den Server gesendet, der sie dann an alle verbundenen Clients zurücksendet. In der Browser-Konsole solltest du die empfangene Nachricht sehen.

Übung

Erweitere die Books-App so, dass sie Websockets verwendet, um alle verbundenen Clients zu benachrichtigen, wenn sich etwas am Bücherbestand ändert (z.B. wenn ein Buch hinzugefügt oder gelöscht wird). Hebe in diesem Fall das geänderte oder neu hinzugekommene Buch farblich hervor.